

第五讲

软件设计原理与结构化设计方法



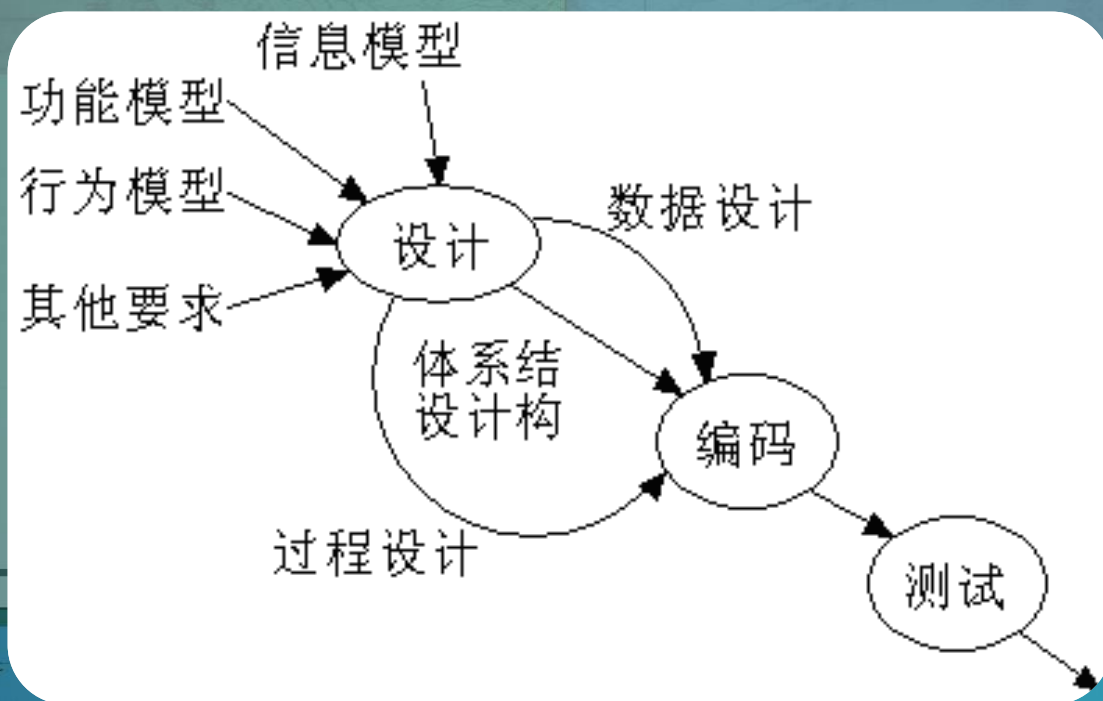
主要内容

- 1、结构化设计与结构化分析的关系
- 2、设计的概念和原理
- 3、模块独立性和启发规则
- 4、表示软件结构的图形工具
- 5、面向数据流的设计方法
- 6、人机界面设计
- 7、过程设计及工具

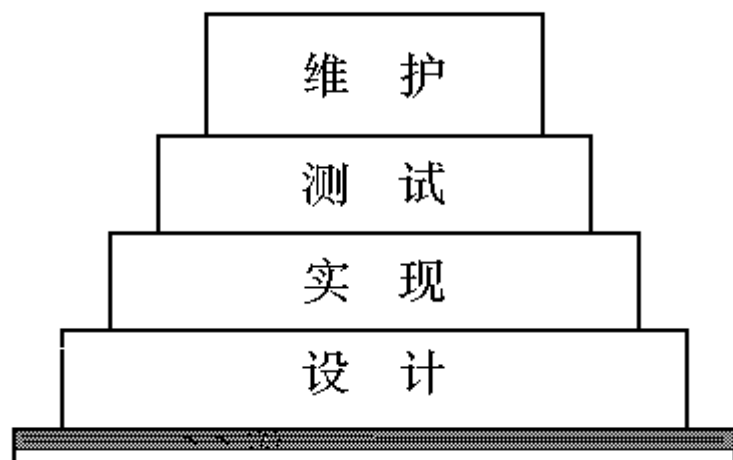


软件设计的重要性

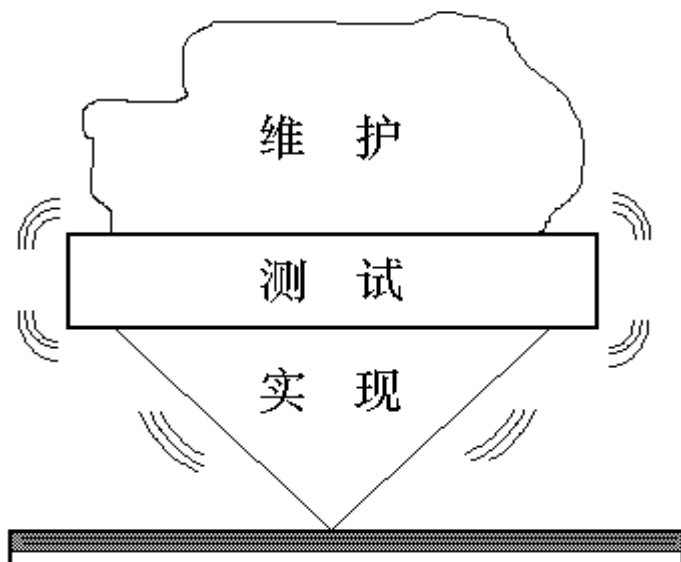
- 软件设计处于软件工程过程的技术核心地位。



软件设计的重要性



有软件设计



没有软件设计

软件设计

- 软件设计的**目标**是设计出所要开发的软件的模型
- 包括**概要设计**和**详细设计**两个阶段：
 - **概要设计**或初步设计：分析软件规格说明，对软件进行功能分解，从而把软件划分为模块，并设计出完成预定功能的模块结构。
 - **详细设计**：详细地设计每个模块，确定完成每个模块功能所需要的算法和数据结构。



主要内容

- 1、结构化设计与结构化分析的关系
- 2、设计的概念和原理
- 3、模块独立性和启发规则
- 4、表示软件结构的图形工具
- 5、面向数据流的设计方法
- 6、人机界面设计
- 7、过程设计及工具



4.2 软件设计的概念和原理

- 模块化
- 抽象
- 逐步求精
- 信息隐藏



4.2.1 模块化

- 模块

- 指由边界元素限定的相邻的程序元素的序列。
- 指整个程序中一些相对独立的程序单元，每个程序单元完成和实现一个相对独立的软件功能。
- 通俗地说就是一些功能独立的程序段。
- 模块具有以下几种基本属性：
 - (1) 接口：指模块的输入与输出。
 - (2) 功能：指模块实现什么功能。
 - (3) 逻辑：描述内部如何实现要求的功能，即内部特性
 - (4) 状态：指该模块的运行环境，即模块的调用与被调用关系。



4.2.1 模块化

- 模块化设计

- 是指把大型软件按照规定的原则划分为一个个较小的、相对独立但又相关的模块的设计方法。

- 实现模块化设计的重要指导思想是：

- 分解
 - 信息隐藏
 - 模块独立性



4.2.1 模块化

- 问题复杂性与工作量关系

- 设问题 x ，表示它的复杂性函数为 $C(x)$ ，解决它所需的工作量函数为 $E(x)$ 。对于问题 $P1$ 和 $P2$ 。如果。

那么，是否可以得出这样的结论：

如果把软件无限细分，那么最后开发软件所需要的工作量就小得可以忽略了。

但是，事实上，影响软件开发工作量的因素还有许多。如，随着模块数目的增加，设计模块之间接口所需要的工作量也将增加。

- $E(P1+P2) > E(P1) + E(P2)$
- 所得结果对于模块化和软件具有重要的意义。

大而
而这



4.2.1 模块化

- 系统模块设计方法有效性的评价标准

- 1、模块可分解性

提供了把问题分解为子问题的系统化机制，能降低整个问题的复杂性。

- 2、模块可组装性

能够把现有的（可重用的）设计构件组装成新系统。

- 3、模块可理解性

可以把一个模块作为一个独立单元（无需参考其它模块）来理解，这样的模块是易于构造和修改的。



4.2.1 模块化

– 4、模块连续性

如果对系统需求的微小修改只导致个别模块的修改，则修改所引起的副作用将最小。

– 5、模块保护性

如果在一个模块内部出现异常时，影响只是局限于模块内部，则错误引起的副作用将最小。



4.2.1 模块化

- 模块化的优点

- 软件结构清晰 → 容易设计和阅读理解;
- 错误局限在相关模块及其接口中 → 容易测试、调试, 提高可靠性;
- 修改和变动只涉及少数几个模块 → 提高可修改性;
- 有助于工程的组织管理 → 有利于合理分工。



4.2 软件设计的概念和原理

- 模块化
- 抽象
- 逐步求精
- 信息隐藏

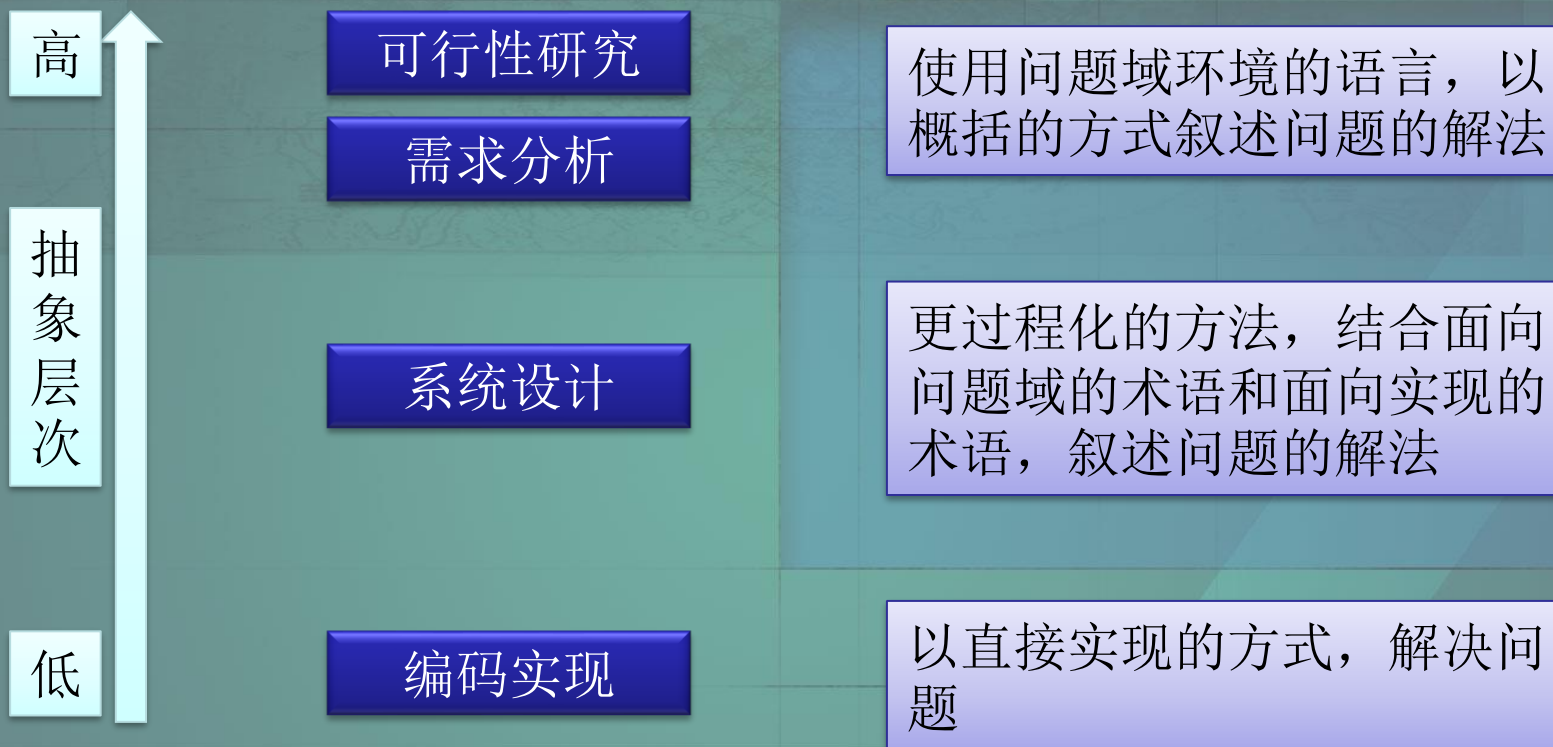


4.2.2 抽象

- **抽象**就是抽出事物的本质特征而暂时不考虑它们的细节的方法，它是人类在认识复杂现象的过程中使用的最强有力的思维工具。
- 处理复杂系统的最重要的有效方法是用**层次的方式**构造和分析它。
- 具体而言：对于一个复杂的动态系统，首先可以用一些高级的抽象概念构造和理解，这些高级概念又可以用一些较低级的概念构造和理解，如此进行下去，直到最低层次的具体元素。



4.2.2 抽象



4.2 软件设计的概念和原理

- 模块化
- 抽象
- 逐步求精
- 信息隐藏



4.2.3 逐步求精

- **逐步求精**是解决复杂问题时采用的基本技术，为了能够集中精力解决主要问题而尽量推迟对问题细节的考虑。
- Miller法则：一个人在任何时候都只能把注意力集中在 7 ± 2 个知识块。
- 通过逐步求精，把一个时期内必须解决的种种问题按优先级排序的技术，它确保每个问题都被解决，而且每个问题都在适当的时候解决。
- **求精实际上是细化过程，和抽象互补。**



4.2.3 逐步求精

- **概括说明：**
- 1、对付复杂问题的最重要方法是抽象；
- 2、因此对一个复杂问题不应该立刻用计算机指令、数字和逻辑符号来表示，而应该用较自然的抽象语句来表示，从而得出抽象程序。
- 3、抽象程序对抽象的数据进行某些特定的运算并用合适的记号（可能是自然语言）来表示；
- 4、对抽象程序做进一步的分解，并进入下一个抽象层次；
- 5、这样的精细化过程一直进行下去，直到程序能被计算机接受为止。这时的程序可能是某种高级语言或机器指令书写的。



4.2 软件设计的概念和原理

- 模块化
- 抽象
- 逐步求精
- 信息隐藏



4.2.4 信息隐藏

- **模块内部的数据与过程**，应该对不需要了解这些数据与过程的模块隐藏起来；只有那些为了完成软件的总体功能而必须在模块间交换的信息，才允许在模块间进行传递。
- “隐蔽”意味着有效的模块化可以通过定义一组独立的模块而实现，这些独立的模块彼此间仅仅交换那些为了完成系统功能而必须交换的信息。
- 这一指导思想的**目的**是为了提高模块的独立性，即当修改或维护模块时减少把一个模块的错误扩散到其他模块中去的机会。



主要内容

- 1、结构化设计与结构化分析的关系
- 2、设计的概念和原理
- 3、模块独立性和启发规则
- 4、表示软件结构的图形工具
- 5、面向数据流的设计方法
- 6、人机界面设计
- 7、过程设计及工具



4.3 模块独立

- **模块独立**是模块化、抽象、逐步求精等概念的直接结果，也是**有效模块设计的基本标准**。
- 模块独立性概括了把软件划分为模块时要遵守的准则，也是判断模块构造是否合理的标准。一般地，坚持模块的独立性是获得良好设计的关键。
- 模块独立性之所以重要，其原因为：
 - 容易开发；
 - 容易测试和维护。
- **模块独立性可以由两个定性标准度量——耦合和内聚。**模块独立性比较强的模块应是高内聚、低耦合的模块。



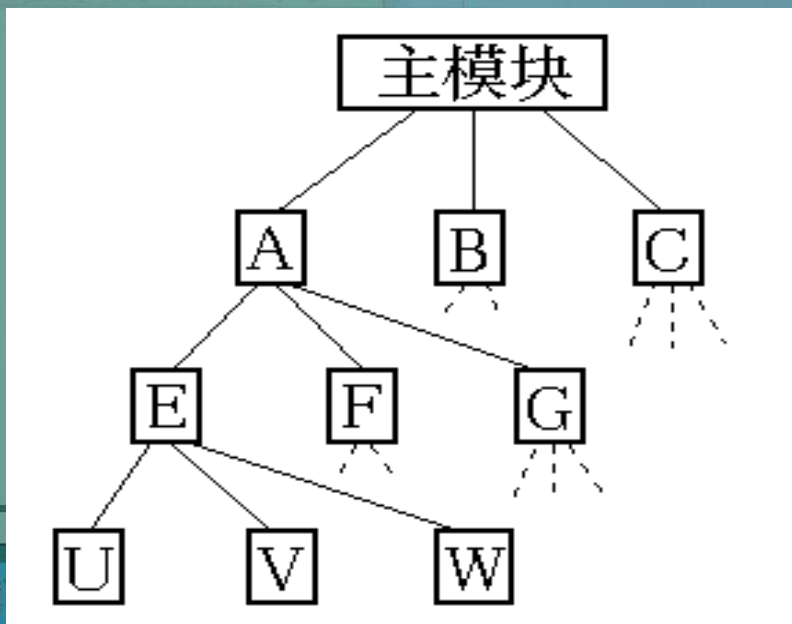
4.3.1 耦合

- **耦合（Coupling）**表示软件结构内不同模块彼此之间相互依赖（连接）的紧密程度，是衡量软件模块结构质量好坏的度量，是对模块独立性的直接衡量指标。
- 耦合强弱取决于模块间接口的复杂程度，进入或访问一个模块的点，以及通过接口的数据。
- 在软件设计中应追求模块间尽可能松散耦合：
 - 在研究、测试或维护任何一个模块时，不需要对系统中的其他模块有很多的了解；
 - 由于模块间联系简单，发生在一处的错误传播到整个系统的可能性就很小。



非直接耦合

- 两个模块之间没有直接关系，它们之间的联系完全是通过主模块的控制和调用来实现的。非直接耦合的模块独立性最强。



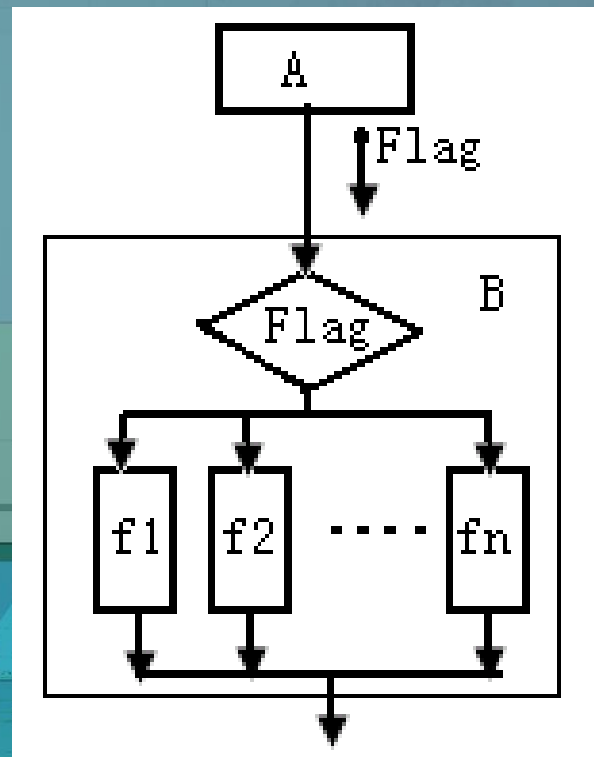
1、数据耦合 Data Coupling

- 如果两个模块彼此间通过参数交换信息，而且交换的信息仅仅是数据，那么这种耦合称为数据耦合。
- 数据耦合是低耦合。系统中必须存在这种耦合，因为只有当某些模块的输出数据作为另一些模块的输入数据时，系统才能完成有价值的功能。



2、控制耦合 Control Coupling

- 如果一个模块通过传送开关、标志等控制信息，明显地控制选择另一模块的功能，就是控制耦合
- 控制耦合往往是多余的，在把模块适当分解之后通常可以用数据耦合代替它。



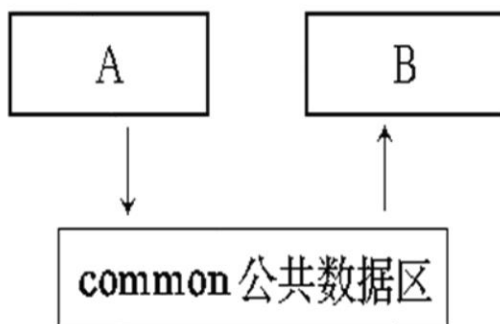
3、特征耦合

- 如果被调用的模块需要使用作为参数传递进来的数据结构中的一部分数据元素时，就出现了特征耦合。
- 在这种情况下，被调用的模块可以处理的数据多于它确实需要的数据，这将导致对数据的访问失去控制。

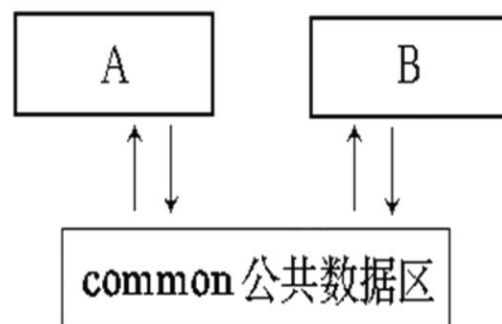


4、公共环境耦合

- 当两个或多个模块通过一个公共数据环境相互作用时，它们之间的耦合称为公共环境耦合（即公用耦合）。
- 公共环境耦合的复杂程度随耦合的模块个数而变化，当耦合的模块个数增加时复杂程度显著增加
- 是可以利用的耦合。如通过文件传递复杂数据。



(a) 松散의公共耦合

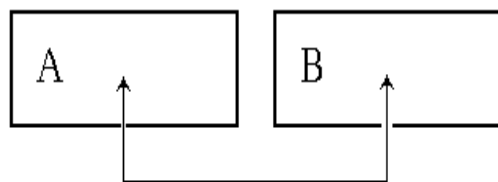


(b) 紧密的公共耦合

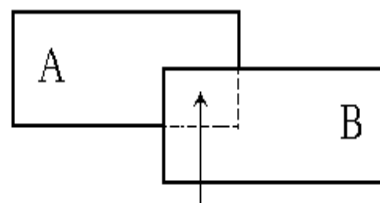


5、内容耦合

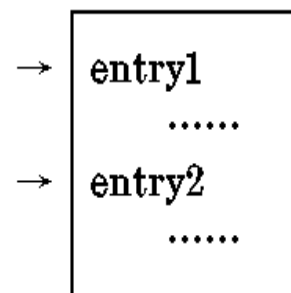
- 最高程度的耦合是内容耦合。
 - (1) 一个模块访问另一个模块的内部数据；
 - (2) 一个模块不通过正常入口而转到另一个模块的内部
 - (3) 两个模块有一部分程序代码重叠（只可能出现在汇编程序中）；
 - (4) 一个模块有多个入口。
- 应该坚决避免使用内容耦合。事实上许多高级程序设计语言已经设计成不允许在程序中出现任何形式的内容耦合。



(a) 进入另一模块内部



(b) 模块代码重迭



耦合是影响模块结构和软件复杂程度的一个重要因素，应该采用如下设计原则：

尽量使用**数据耦合**，少用**控制耦合**和**特征耦合**，限制**公共环境耦合**的范围，完全不用**内容耦合**。



4.3.2 内聚

- 内聚标志着一个模块内部各个元素间彼此结合的紧密程度。简单地说，理想内聚的模块只做一件事情。
- 设计时应该力求做到高内聚，通常中等程度的内聚也是可以采用的，而且效果和高内聚相差不多。但是，坚决不要使用低内聚。
- 内聚和耦合是密切相关的，模块内的高内聚往往意味着模块间的低耦合。内聚和耦合都是进行模块化设计的有力工具。实践表明，内聚更重要，应该把更多注意力集中到提高模块的内聚程度上。





低内聚

中内聚

高内聚

1、功能内聚Functional Cohesion

- 如果模块内所有处理元素属于一个整体，完成一个单一的功能，则称为功能内聚。功能内聚是最高程度的内聚。
- 如：依据雇员生日来计算年龄的子程序就是功能内聚性的，因为它只完成一项工作。



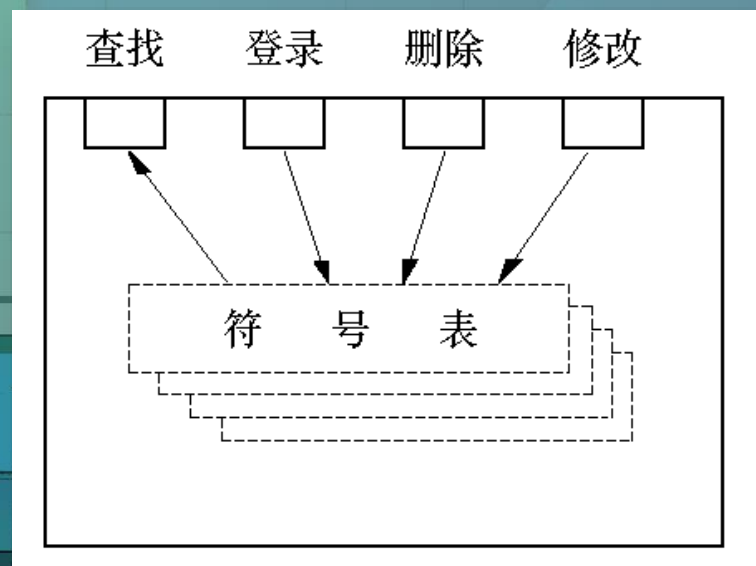
2、顺序内聚

- 如果一个模块内的处理元素和同一个功能密切相关，而且这些处理必须顺序执行（通常一个处理元素的输出数据作为下一个处理元素的输入数据），则称为顺序内聚。
- 如：假设有一个按给出的生日计算雇员年龄、退休时间的子程序，如果它是利用所计算的年龄来确定雇员将要退休的时间，那么它就具有顺序内聚性。



*信息内聚 Informational Cohesion

- 这种模块完成多个功能，各个功能都在同一数据结构上操作，每一项功能有一个唯一的入口点。这个模块将根据不同的要求，确定该执行哪一个功能。这个模块的所有功能都是基于同一个数据结构（如图中所示的符号表）。
- 信息内聚模块可以看成是多个功能内聚模块的组合，并且达到信息的隐蔽。即把某个数据结构、资源或设备隐蔽在一个模块内。



3、通信内聚Communication Cohesion

- 如果一个模块中所有处理元素都使用同一个输入数据和（或）产生同一个输出数据，称为通信内聚。通常，通信内聚模块是通过数据流图来定义的。
- 如：假设有一个按给出的生日计算雇员年龄、退休时间的子程序，如果它是分别计算年龄和退休时间的，但使用相同生日数据，那它就只具有通信内聚性。
- 如：有一个打印总结报告，并在完成后重新初始化传进来的总结数据的子程序，这个子程序具有通信内聚性，因为这两个操作仅仅是由于它们使用了相同的数据才联系在一起。



4、过程内聚

- 如果一个模块内部的处理元素是相关的，而且必须以特定次序执行，则称为**过程内聚**。把流程图中某一部分划出组成模块，就得到过程内聚模块。
- **过程内聚与顺序内聚的区别**主要在于：顺序内聚中是数据流从一个处理元流到另一个处理元，而过程内聚中是控制流从一个动作流向另一个动作。
- 如：假设有一个子程序，它读取雇员的名字，然后是地址，最后是它的电话号码。这种顺序之所以重要，仅仅是因为它符合用户的要求，用户希望按这种顺序进行屏幕输入。



5、时间内聚 Classical Cohesion

- 这种模块大多为多功能模块，但模块的各个功能的执行与时间有关，通常要求所有功能必须在同一时间段内执行。例如初始化模块和终止模块。

紧急故障处理模块

1. 关闭文件

2. 报警

3. 保留现场



6、逻辑内聚 Logical Cohesion

- 这种模块把几种相关或相似的功能组合在一起，每次被调用时，由传送给模块的判定参数来确定该模块应执行哪一种功能。
- 逻辑内聚的模块各成分在功能上并无关系，即使局部功能的修改有时也会影响全局，因此这类模块的修改也比较困难。
- 如：将一套生成不同输出报告的打印函数被安排在同一个模块中；
- 如：从不同外设读取输入数据。



6、逻辑内聚 Logical Cohesion

- 如：一个子程序将打印季度开支报告、月份开支报告和日开支报告。具体打印哪一个，将由传入的控制标志决定。它的内部逻辑是由输进去的外部控制标志决定的。
- 改进原来的子程序，让它根据传送去控制标志来调用这三个子程序之一。调用子程序将只有调用代码而没有自己的计算代码，因而具有功能内聚性。而三个被调用的子程序也显然是功能内聚性的。这个只负责调用其它子程序的子程序也是一个事务处理中心。最好用带有“调度”或“控制”等字眼的词来给事务处理中心命名，以表示它只负责调度，而本身并不做任何工作。



7、偶然内聚Coincidental Cohesion

- 当模块内各部分之间没有联系，或者即使有联系，这种联系也很松散，称这种模块为偶然内聚模块，它是内聚程度最低的模块。
- 如，发现一组语句在多处出现，为了节约空间将这些语句作为一个模块设计。
- 很有可能是出于偶然，而不是基于任何想法或设计。





4.4启发规则

- 改进软件结构提高模块独立性

- 审查分析设计出的软件初步结构，通过模块分解和合并，力求降低耦合提高内聚

- 模块规模应该适中

- 过大的模块是由分解不充分造成的，但进一步分解不应降低模块独立性；过小的模块开销大于有效操作，而且还造成系统接口复杂

- 深度、宽度、扇出和扇入应适中

- 模块的作用域应该在控制域之内

- 力争降低模块接口的复杂程度

- 设计单入口、单出口的模块

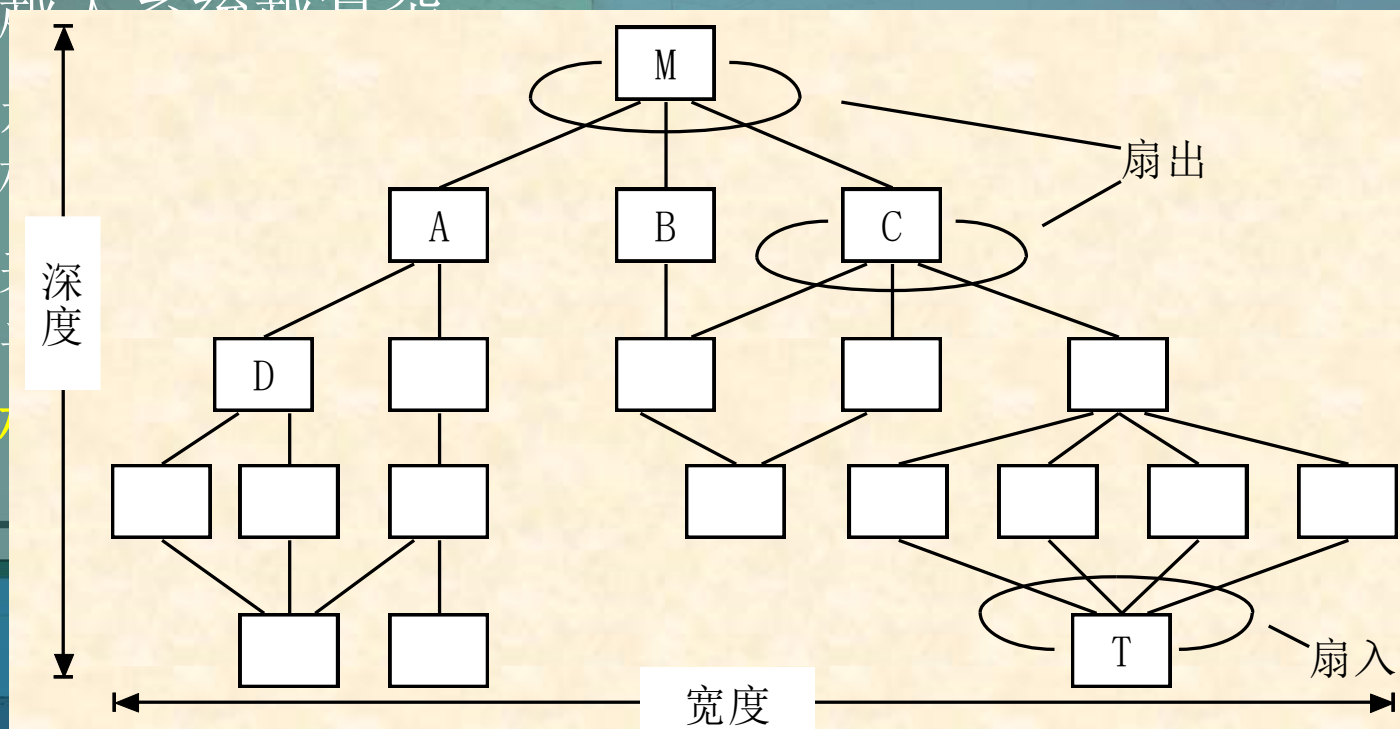
- 模块功能应该可以预测



扇入扇出与深度宽度

- **深度**表示软件结构中控制的层数，能够粗略地标志一个系统的大小和复杂程度；
- **宽度**是软件结构中同一个层次上的模块总数的最大值，宽度越大系统越复杂；

- **扇出**表示一个模块直接控制的模块个数
- **扇入**表示有多少个模块直接控制一个模块
- **扇入共享**表示多个模块共享一个公共模块
- **违背**表示违背了软件结构设计原则



扇入扇出与深度宽度

- 一个系统的大小和系统的复杂程度在一定程度上可以用深度和宽度表示。
- 大量的系统研究表明，
 - 高层模块应有较高的扇出
 - 低层模块特别是底层模块应有较高的扇入。

（扇入越大，表示该模块被更多的上级模块共享。多个扇入入口相同，这就避免了程序的重复，因此希望扇入高一点。但过多又可能是把许多不相关的功能硬凑在一起，形成通用模块，这样的模块必然是低内聚的。）



模块的作用域与控制域

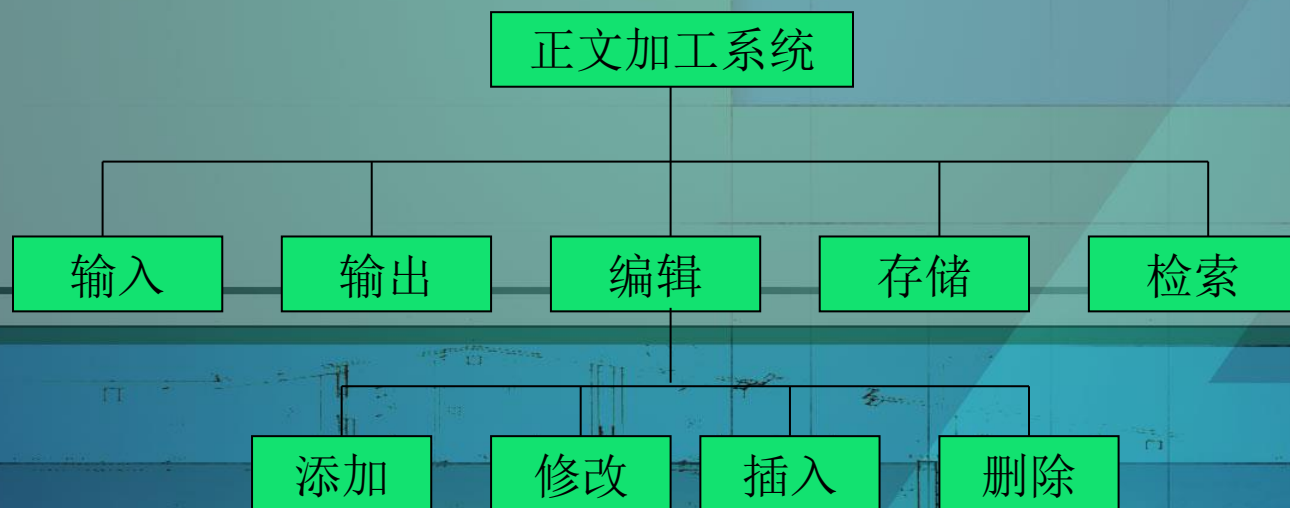
- **模块作用域**定义为受该模块内一个判定影响的所有模块的集合；**模块的控制域**定义为模块本身以及所有直接或间接从属于它的模块。
- 一个模块的作用范围应在其控制范畴之内，且条件判定所在的模块应与受影响的模块在层次上尽量靠近。如果在设计过程中，发现模块作用范围不在其控制范围之内，可用以下方法加以改进：
 - （1）上移判断点。使该判断的层次升高，以扩大它的控制范围。
 - （2）下移受判断影响的模块。将受判断影响的模块下移到判断所在模块的控制范围内。



4.5表示软件结构的图形工具

• 1、层次图

- 层次图（也称H图）是在设计阶段最常使用的图形工具之一，它常用于描绘软件的层次结构
- 层次图中的每个方框代表一个模块，方框间的连线表示模块间的调用关系。



• 2、HIPO图

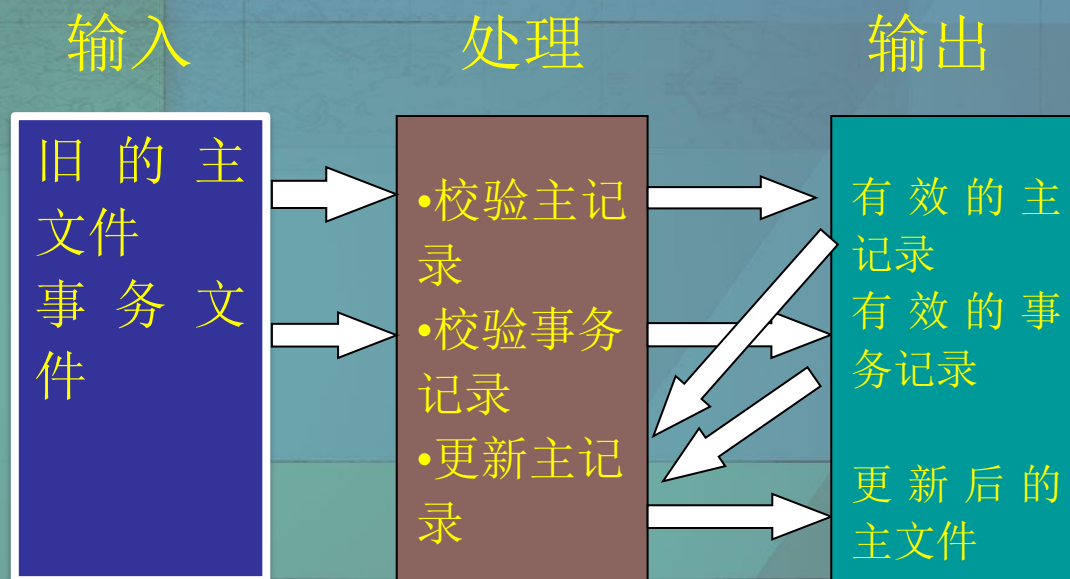
- HIPO图实际上由H图和IPO图两部分组成
- H图即层次图，为了能使HIPO图具有可跟踪性，在H图里除了最顶层的方框之外，每个方框都加了编号

H 图



- 和H图中的每个方框相对应，有一张IPO图描述这个方框代表的模块的处理过程。IPO图能够方便地描述数据输入、数据处理和数据输出之间的关系。

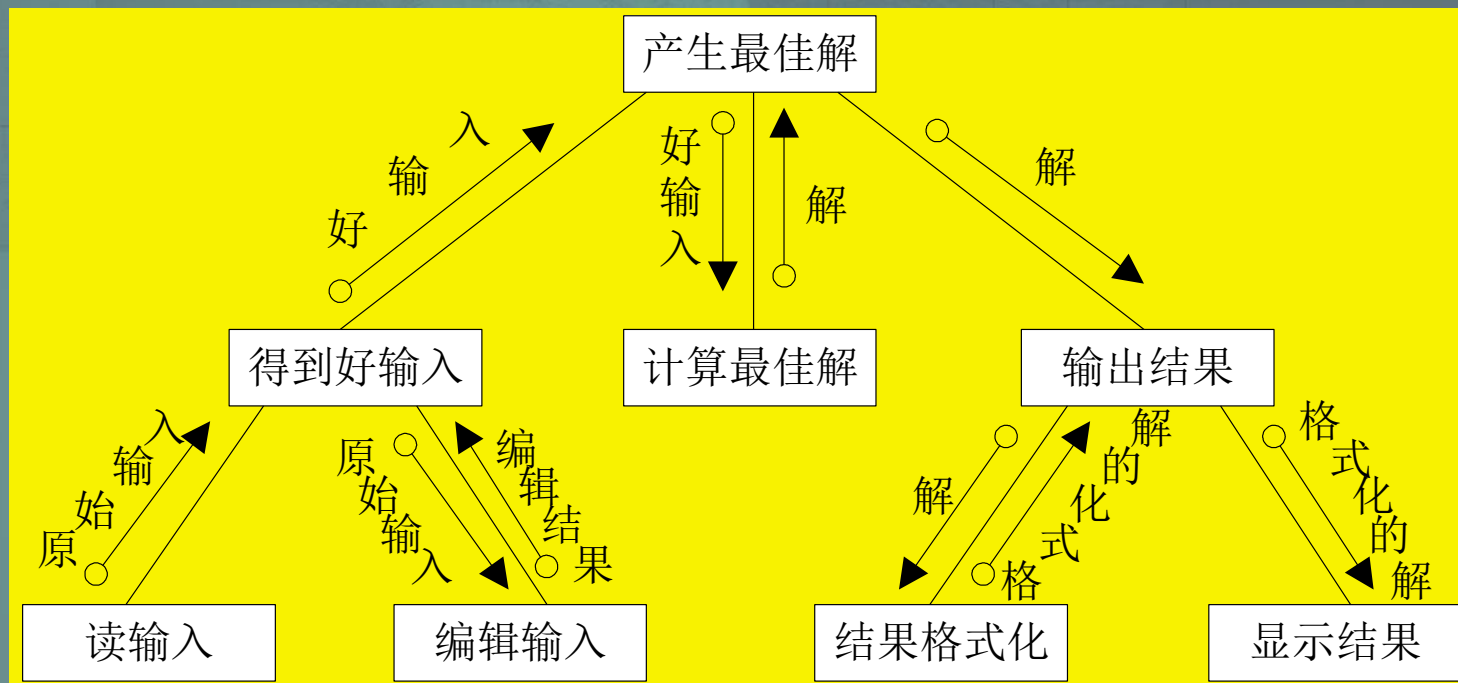
- 1、在左边的框中列出有关的输入数据
- 2、在中间的框中列出主要的处理，处理框中列出的处理次序暗示了执行的次序
- 3、在右边的框中列出产生的输出数据
- 4、用类似向量符号的粗大箭头清楚地指出数据通信的情况

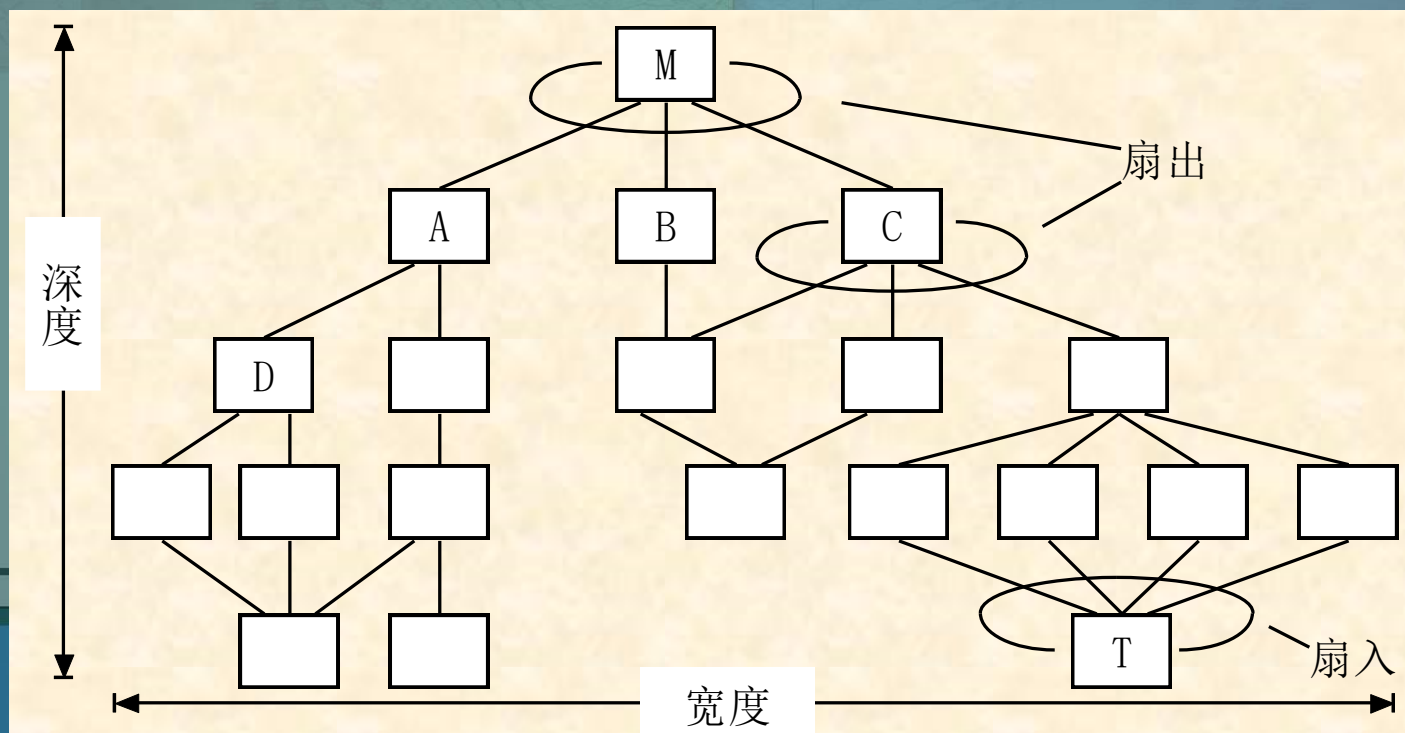
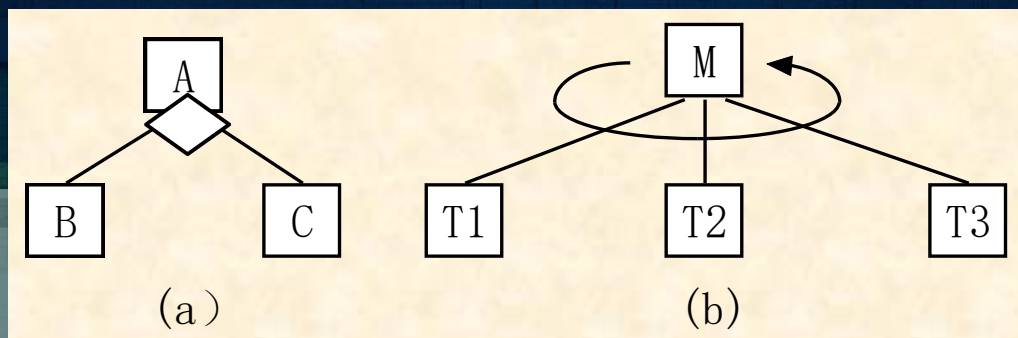


3、软件结构图

- 软件结构图（SC）与层次图类似，是进行软件结构设计的另一个有力工具。
- 具体内容包括：
 - 1、模块：方框，并用体现功能的名字标识；
 - 2、模块的控制关系：单向箭头或直线连接；
 - 3、模块间的信息传递：可用带有注释的短箭头表示模块调用过程中来回传递的信息。
 - 4、选择调用和循环调用
 - 5、结构图的形态特征（宽度、深度、扇入、扇出）





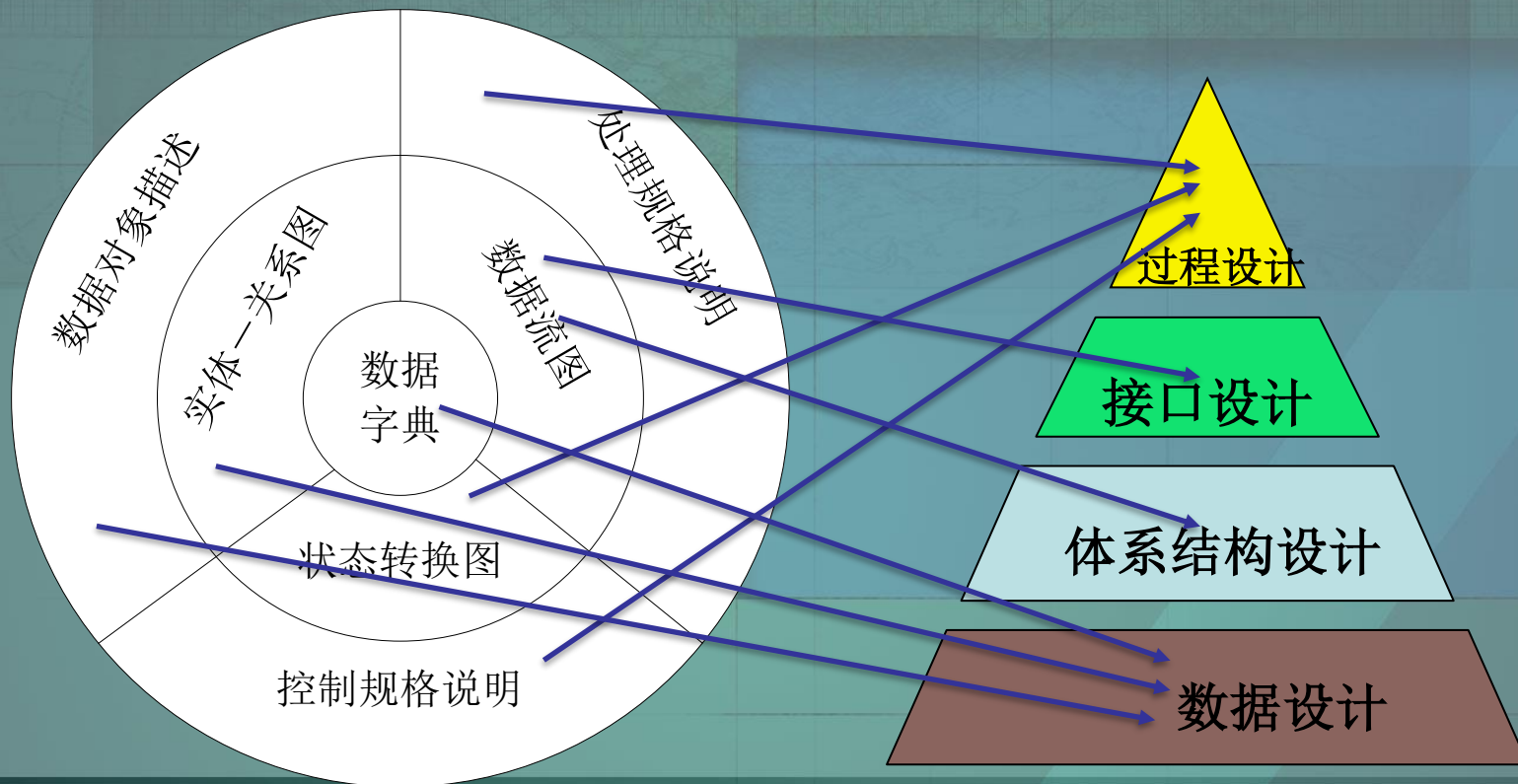


主要内容

- 1、结构化设计与结构化分析的关系
- 2、设计的概念和原理
- 3、模块独立性和启发规则
- 4、表示软件结构的图形工具
- 5、面向数据流的设计方法
- 6、人机界面设计
- 7、过程设计及工具



4.1 结构化设计与结构化分析的关系



4.5 面向数据流的设计

- 面向数据流的设计方法的目标是给出设计软件结构的一个系统化的途径。
- 需求分析阶段，通常用数据流图描绘信息在系统中加工和流动的情况。
- 面向数据流的设计方法定义了一些不同的“映射”，利用这些映射可以把数据流图变换成软件结构。
- 任何软件系统都可以用数据流图表示，所以面向数据流的设计方法理论上可以设计任何软件的结构。

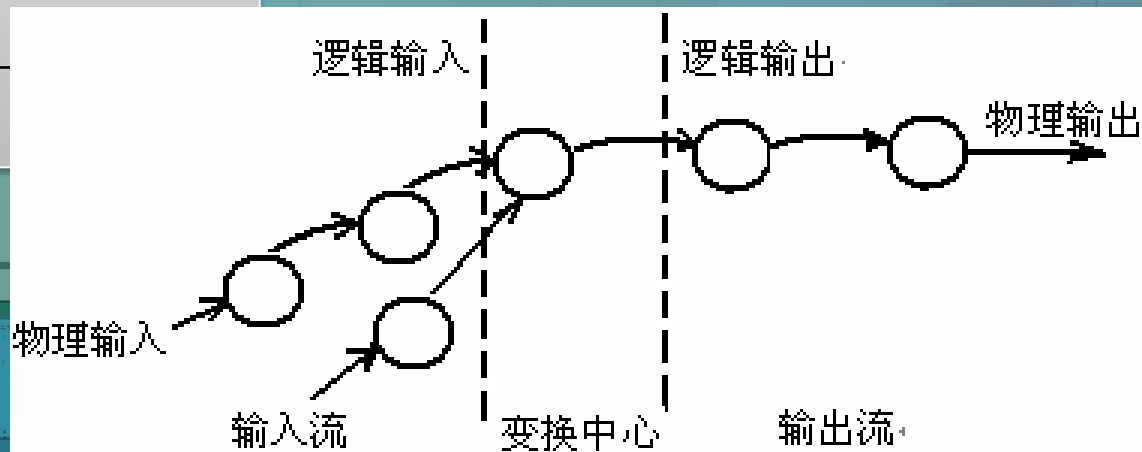
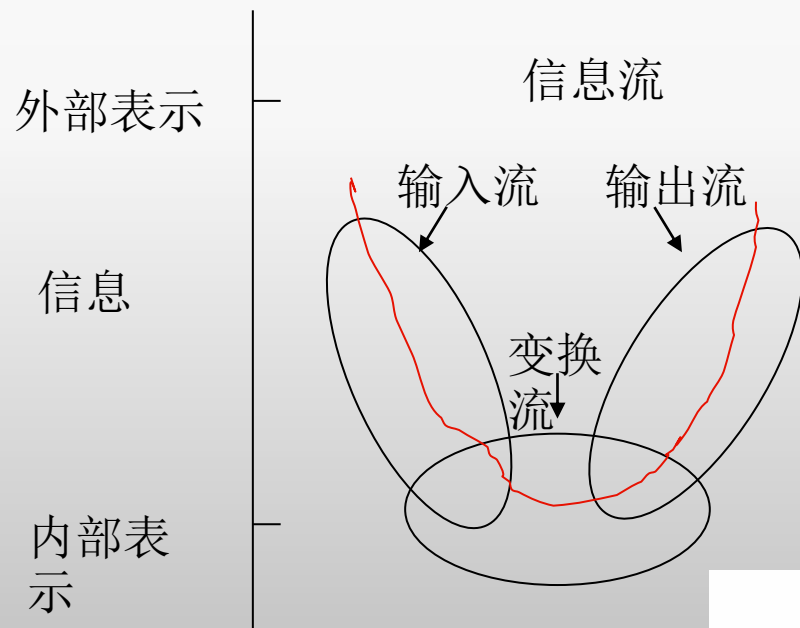


4.5.1 数据流的类型

- 结构化设计的目的是要把数据流图映射成软件结构，而根据数据流图的类型又确定了映射方法。
- **1、变换型数据流图**
 - 根据基本系统模型，信息通常以“外部世界”的形式进入软件系统，经过处理以后再以“外部世界”的形式离开系统
 - 信息沿输入通路进入系统，同时由外部形式变换成内部形式，进入系统的信息通过变换中心，经加工处理以后再沿输出通路变换成外部形式离开软件系统
 - 当数据流图具有这些特征时，就叫做变换流。
 - **原则上可以将所有的信息流都可以归结为这一类**



1、变换型数据流

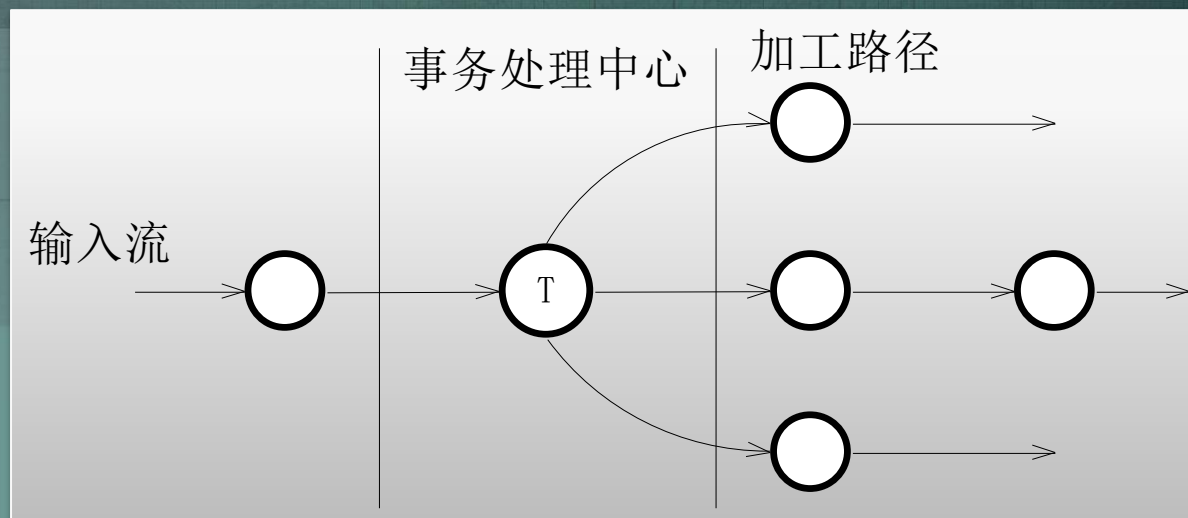


2、事务型数据流

- 当数据流图“以事务为中心”，也就是说，**数据沿输入通路到达一个处理T，这个处理根据输入数据的类型在若干个动作序列中选出一个来执行。**这类系统的特征，是具有在多种事务中选择执行某种事务的能力。
- 事务型结构由至少一条接受路径、一个事务中心和若干条动作路径组成。这类数据流应该划为一类特殊的数据流，称为事务流。



2、事务型数据流

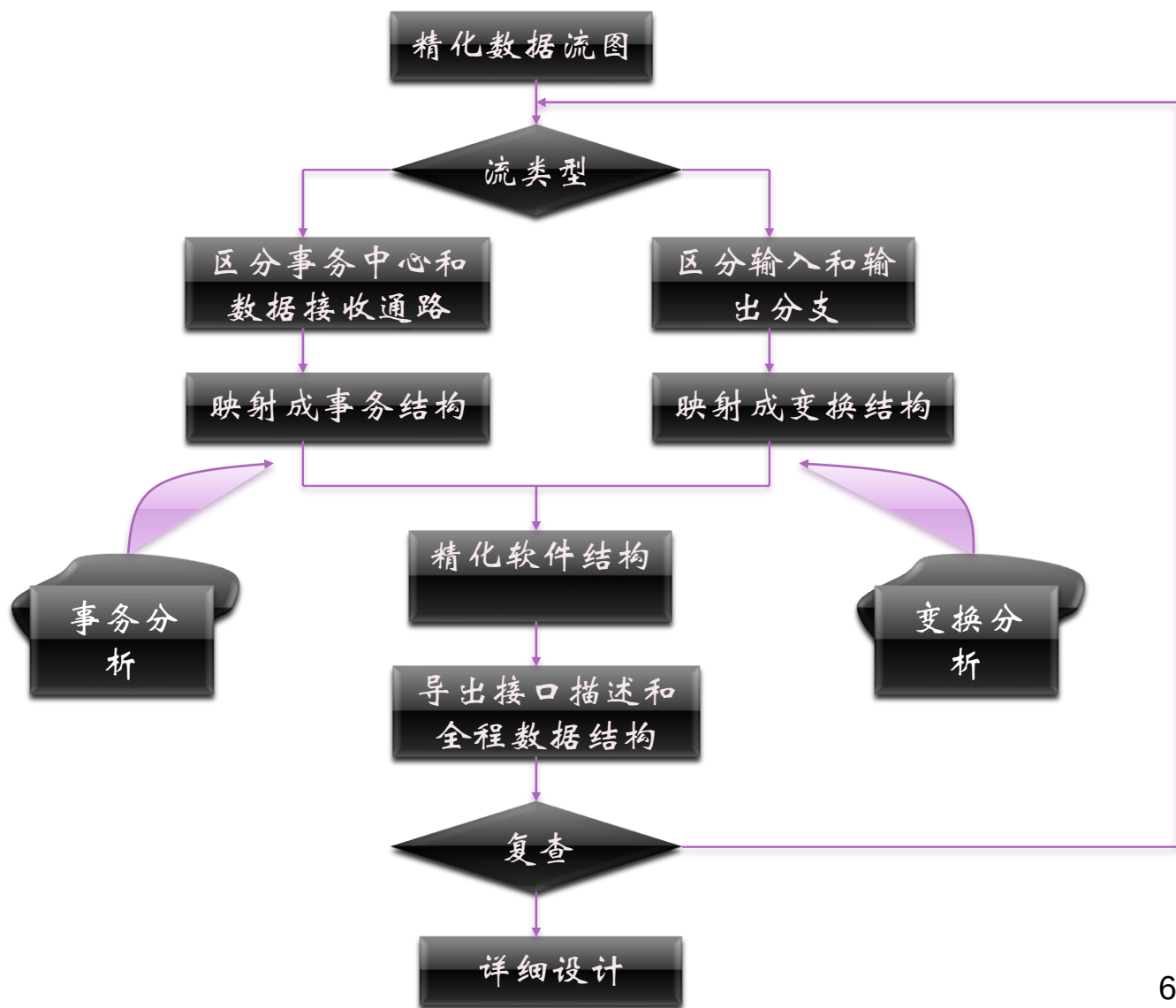


图中的处理T称为事务中心，它完成下述任务：

- (1) 接收输入数据。
- (2) 分析每个事务，确定其类型。
- (3) 根据事务选择一条活动通路。



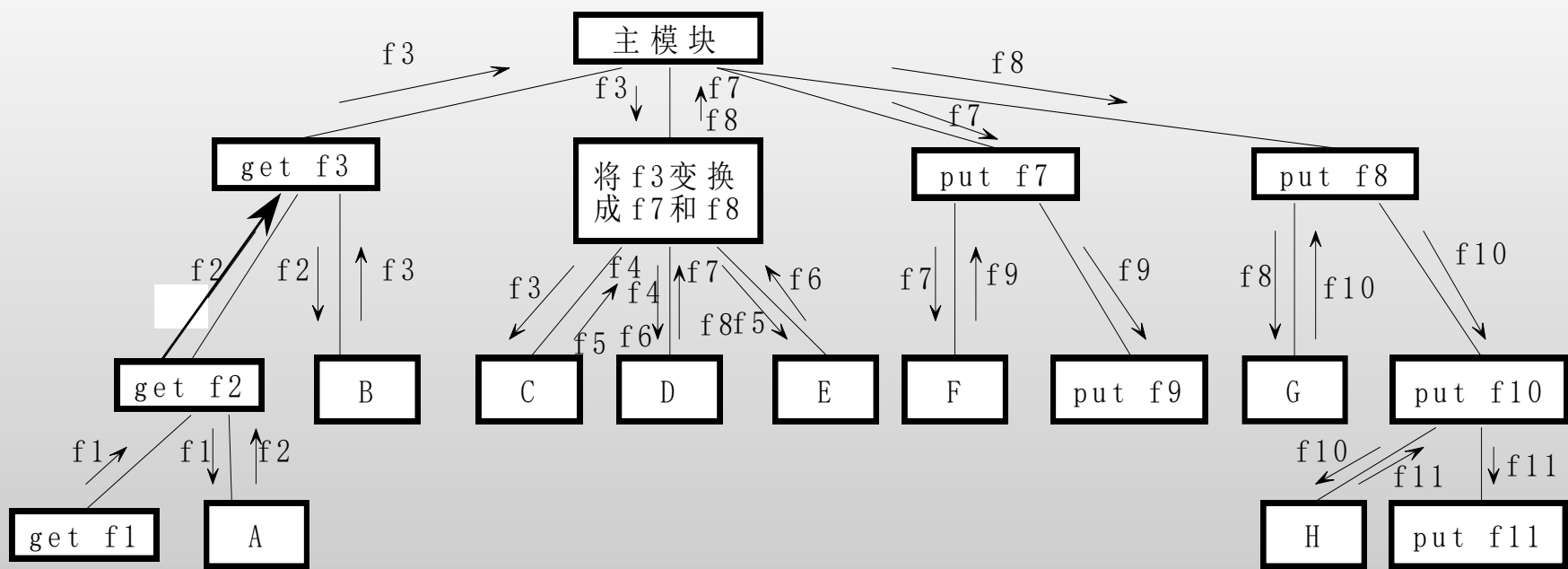
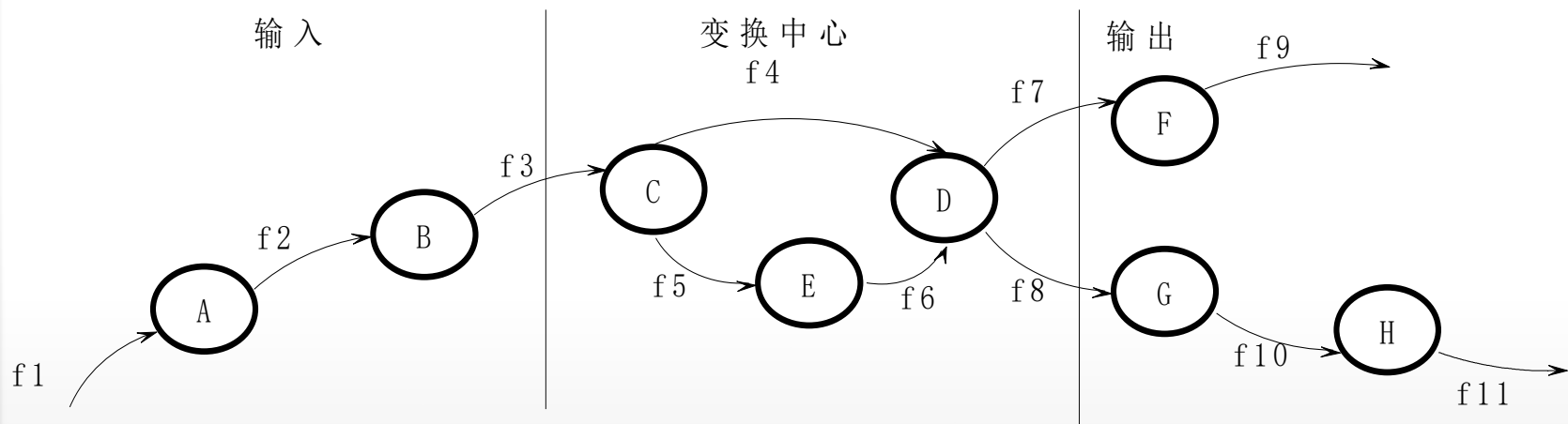
4.5.2面向数据流的设计过程步骤



1、变换分析

- 变换流的设计是将数据流图到程序结构图的转换
- 1. 确定数据流图中的变换中心、逻辑输入和逻辑输出。
- 2. 设计软件结构的顶层和第一层。
- 3. 设计中、下层模块。
 - 1) 输入模块的下属模块的设计
 - 2) 输出模块的下属模块的设计
 - 3) 变换模块的下属模块的设计
- 4. 优化。

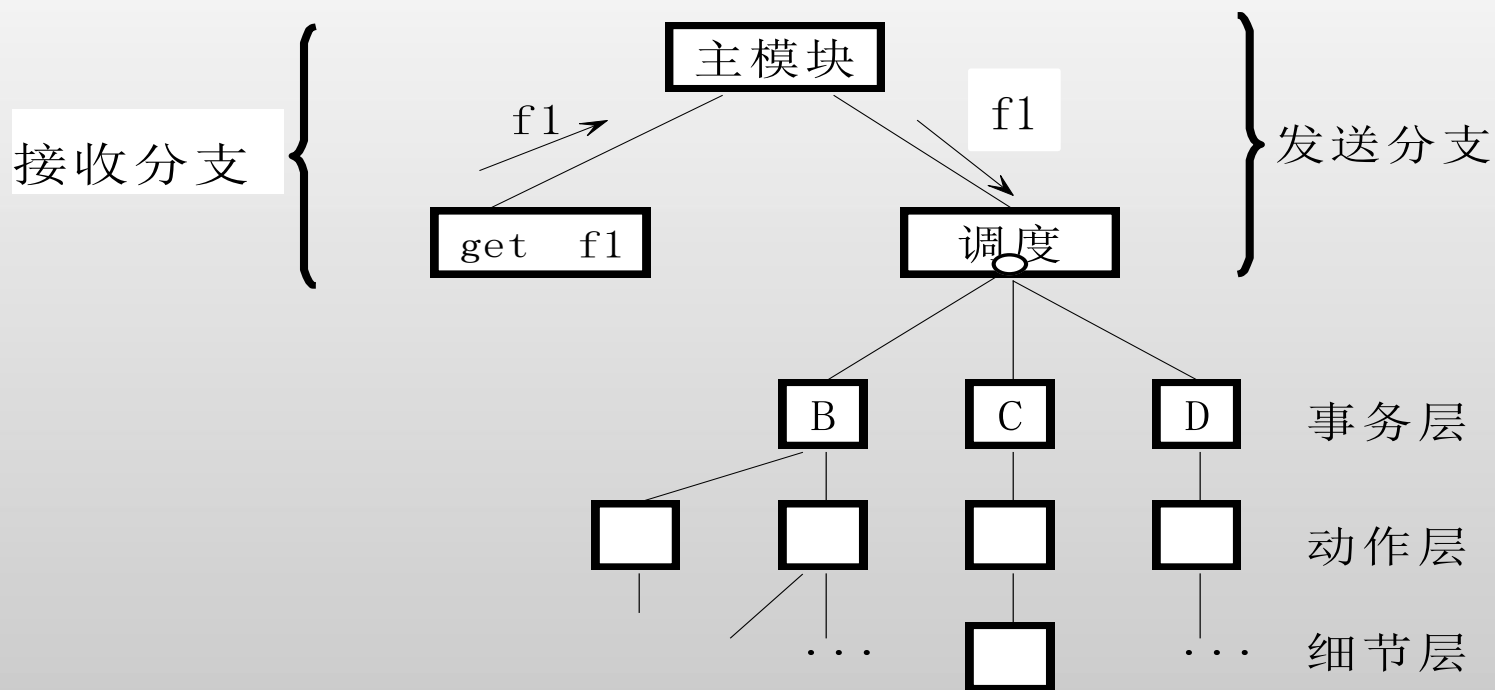
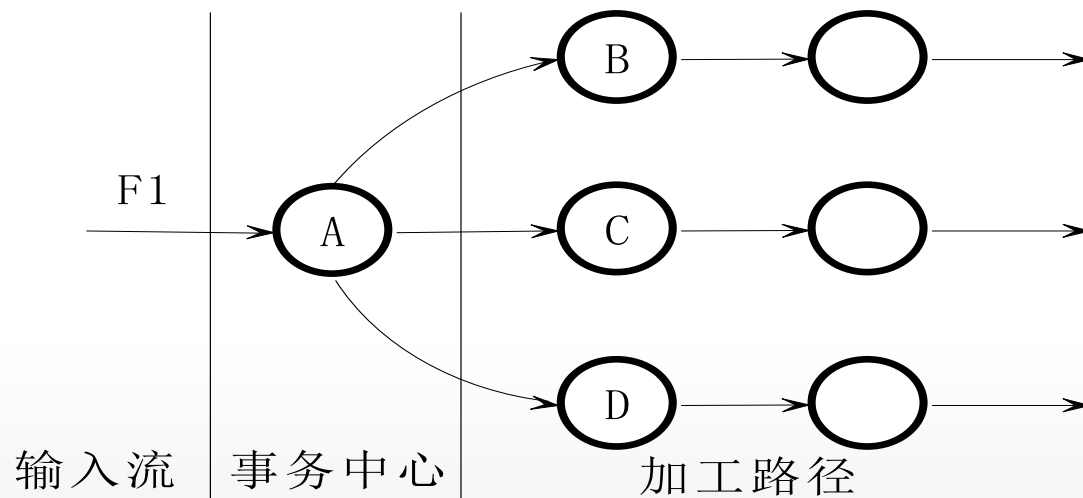




2、事务分析

- 1. 确定数据流图中的事务中心和加工路径。
 - 当数据流图中的某个加工具有明显地将一个输入数据流分解成多个发散的输出数据流时，该加工就是**事务中心**。从事务中心辐射出去的数据流为各个加工路径
- 2. 设计软件结构的顶层和第一层。
 - 由事务流映射成的软件结构包括一个接收分支和一个发送分支。映射**接收分支**时，从事务中心的边界开始，把沿着接收流通路的处理映射成模块。**发送分支**的结构包含一个调度模块，它控制下层的所有活动模块，然后把数据流图中的每个活动流通路映射成与它的流特征相对应的结构
- 3. 进行事务结构中、下层模块的设计、优化工作





概要设计的后期处理

- (1) 为每个模块提供一份接口说明
- (2) 为每个模块写一份处理说明
- (3) 给出设计约束或限制
- (4) 数据结构说明
- (5) 进行设计评审
- (6) 设计优化



4.6 详细设计

- 详细设计以总体设计阶段的工作为基础的，但又不同于总体设计，主要表现为以下两个方面：
 - 在概要设计阶段，数据项和**数据结构**以比较抽象的方式描述，而详细设计阶段则应在此基础上给出足够详细描述。
 - **详细设计要提供关于算法的更多的细节**，例如：总体设计可以声明一个模块的作用是对一个表进行排序，详细设计则要确定使用哪种排序算法。在详细设计阶段为每个模块增加了足够的细节后，程序员才能够以相当直接的方式进行下一阶段的编码工作。



- 详细设计的任务
 - 确定每个模块的算法
 - 确定每一个模块的数据组织
 - 为每个模块设计一组测试用例
 - 编写详细设计说明书



4.7 过程设计

- 过程设计应当在数据设计、体系结构设计和接口设计完成之后进行，它是详细设计阶段应该完成的主要任务。
- 它是一种设计程序的技术，它采用自顶向下逐步求精的设计方法和单入口单出口的控制结构。
- 过程设计的目标不仅仅是逻辑上正确地实现每个模块的功能，更重要的是设计出的处理过程应该尽可能简明易读。

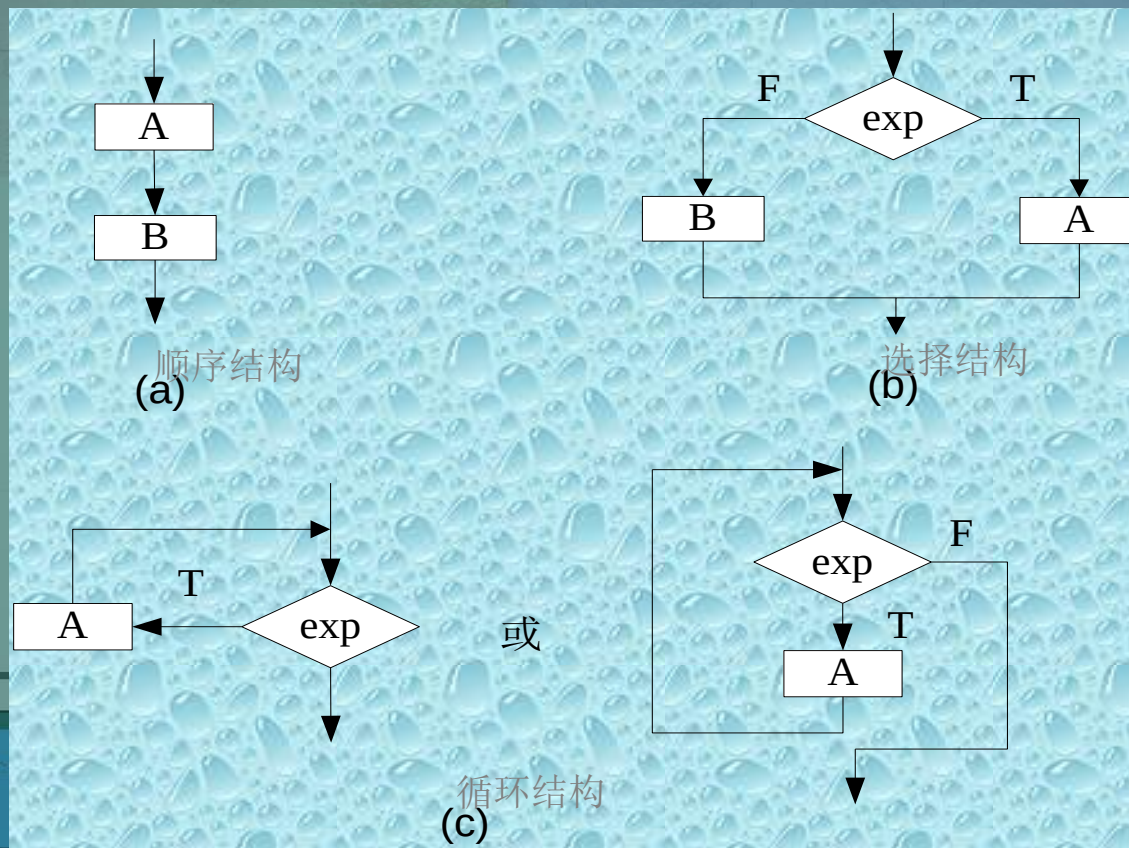


1、过程设计方法（程序设计方法）

- 自顶向下、逐步求精
 - 在需求分析、总体设计中，都采用了自顶向下、逐层细化的方法。
- 使用三种基本控制结构构造程序
 - 任何程序都可由**顺序、选择及重复**三种基本控制结构构造，为自顶向下、逐步求精的设计方法提供了具体的实施手段。
 - （1）用顺序方式对过程分解，确定执行顺序；
 - （2）用选择方式，确定某个部分的执行条件；
 - （3）用循环方式，确定某个部分进行重复的开始和结束条件。



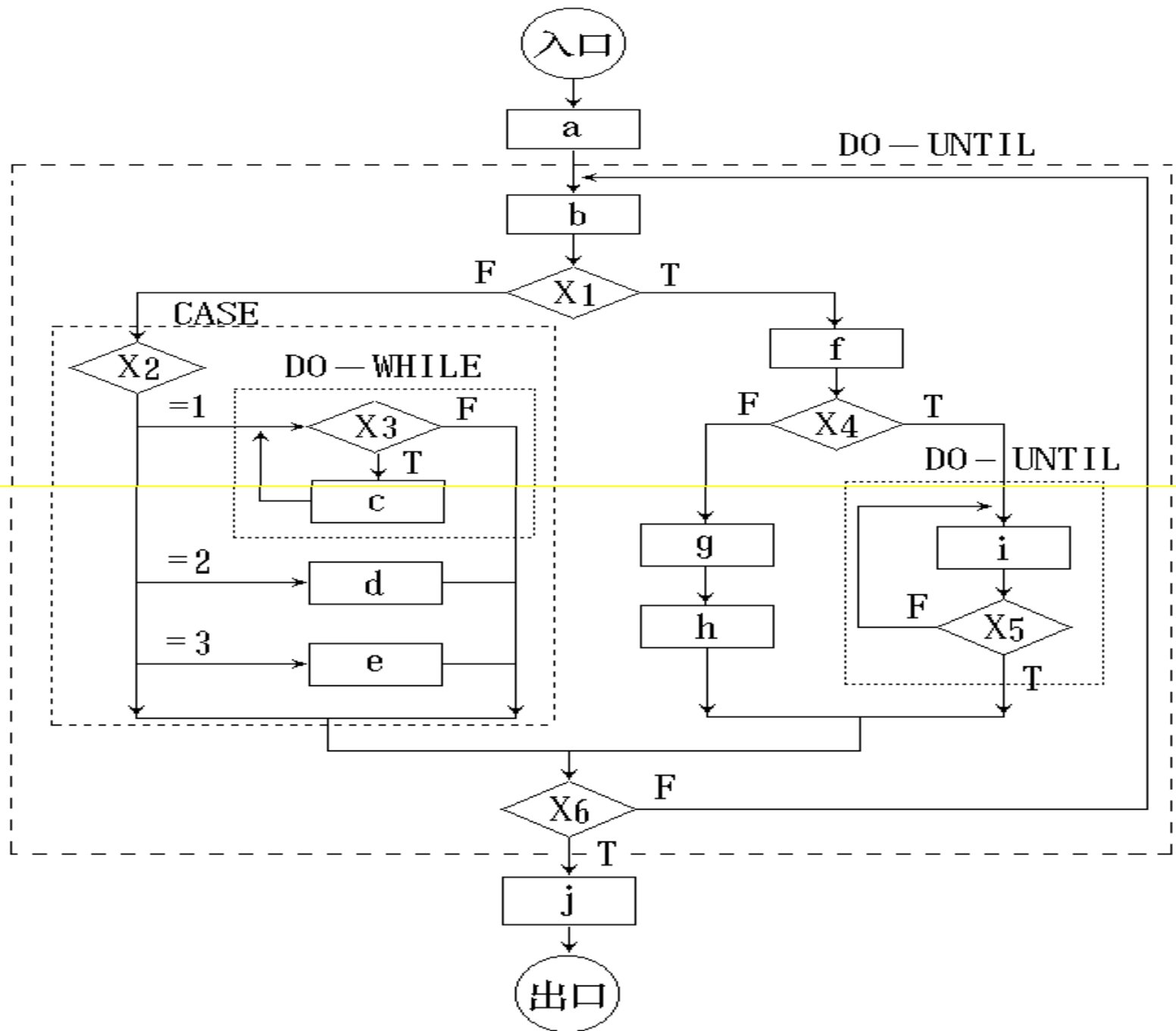
- 三种基本的控制结构



2、过程设计工具

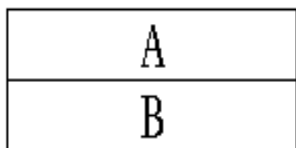
- 描述程序处理过程的工具称为过程设计的工具，包括图形、表格和语言三类。
- 通过过程设计工具提供了对设计无歧义的描述，也就是应该能指明控制流程、处理功能、数据组织以及其他方面的实现细节，从而在编码阶段能把对设计的描述直接翻译为程序代码。
- 常用的过程设计工具：程序流程图、N-S图、PAD图、判定表和判定树、PDL语言。



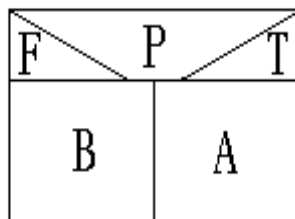


N-S图

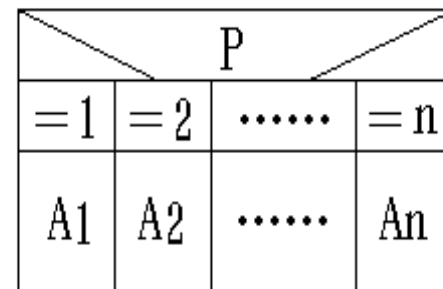
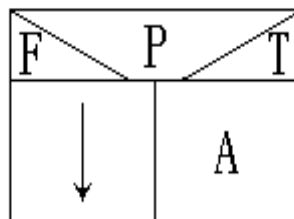
- N-S图也称为盒图。由于盒图没有箭头，不允许随意转移控制，是一种不违背结构程序设计精神的图形工具。



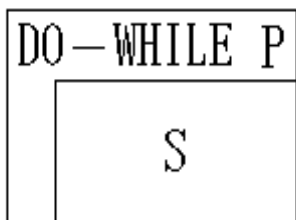
① 顺序型



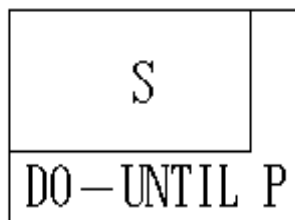
② 选择型



⑤ 多分支选择型
(CASE型)



③ WHILE重复型



④ UNTIL重复型

a							
b							
T \ X1			F				
f			X2				
T \ X4			F	= 1	= 2	= 3	
i		X5	g	DO — WHILE		d	e
DO — UNTIL			h	X3	c		
DO — UNTIL X6							
j							

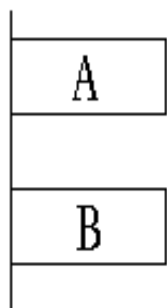
N-S图

- 优点：
 - 功能域（一个特定控制结构的作用域）明确，可以从盒图上一眼就看出来
 - 不可能任意转移控制
 - 很容易确定局部和全局数据的作用域
 - 很容易表现嵌套关系，也可以表达模块的层次结构

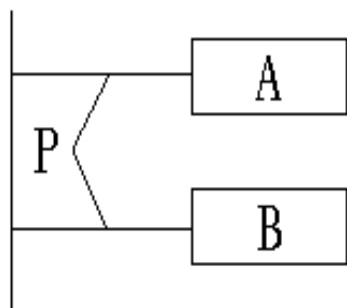


PAD图

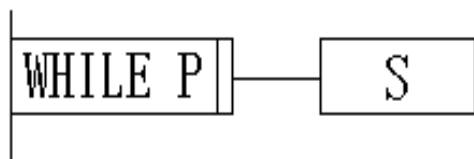
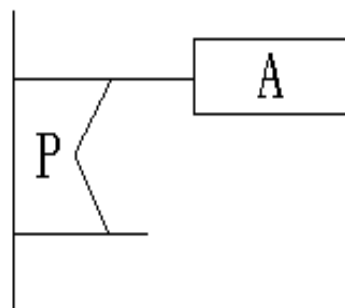
- 问题分析图（Problem Analysis Diagram, PAD）由日立公司于1979年提出。
- 是用结构化程序设计思想表现程序逻辑结构的图形工具。



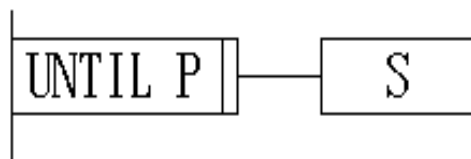
① 顺序型



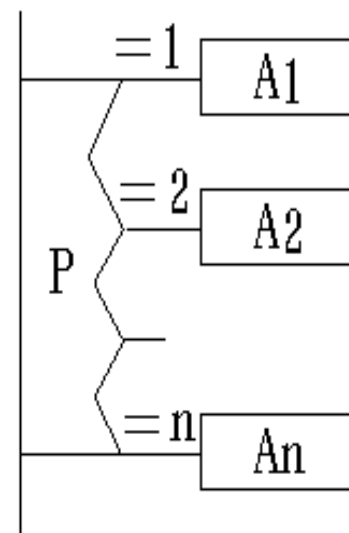
② 选择型



③ WHILE 重复型

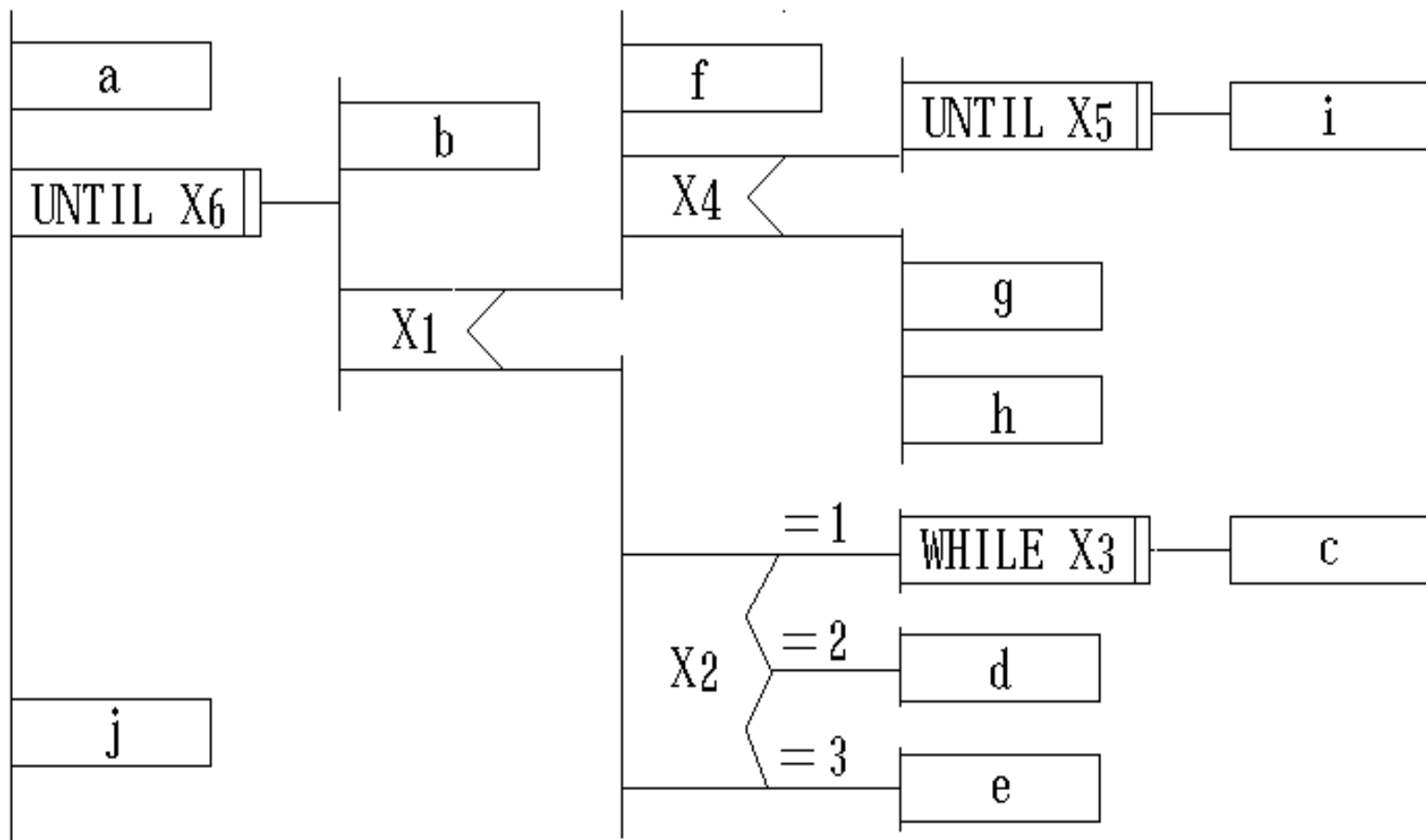


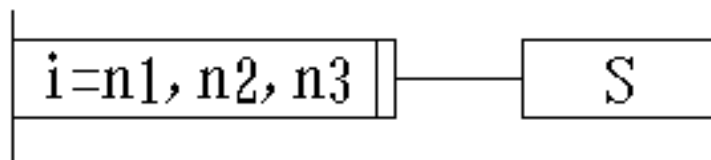
④ UNTIL 重复型



⑤ 多分支选择型
(CASE 型)

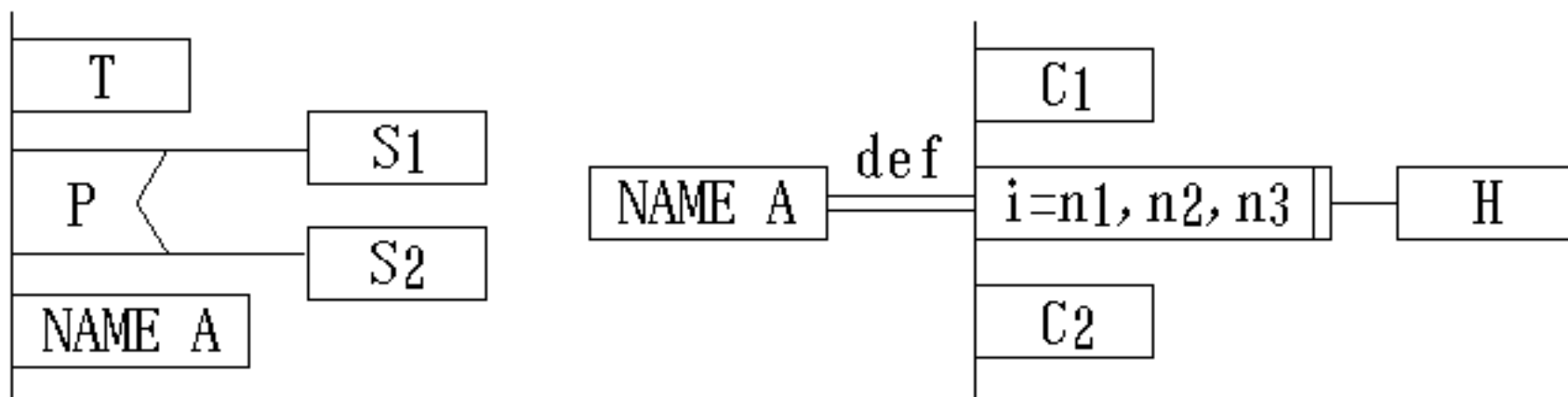






(a) FOR 重复型

其中, $i=n1, n2, n3$ 表示
 $i:=n1$ to $n2$ step $n3$



(b) def 格式

PAD图

- 优点：
 - 以问题分析图为基础，按照一个机械的变换规则就可编写计算机程序
 - 问题分析图有着逻辑结构清晰，图形化标准化与人们所熟悉的控制流程图比较相似等优点
 - 更重要的，它引导设计人使用结构化程序设计方法，从而提高程序的质量。
 - 自上而下、自左向右顺序执行，易读、易懂；
 - 最左面的竖线是程序的主线，即第一层结构。随着程序层次的增加，PAD图逐渐向右延伸



判定表和判定树

- 判定表及判定树主要表达算法中所包含的多重嵌套的条件选择与应做动作之间的对应关系。
- 判定表由四部分组成：

条件定义

条件组合

动作定义

条件组合下的动作



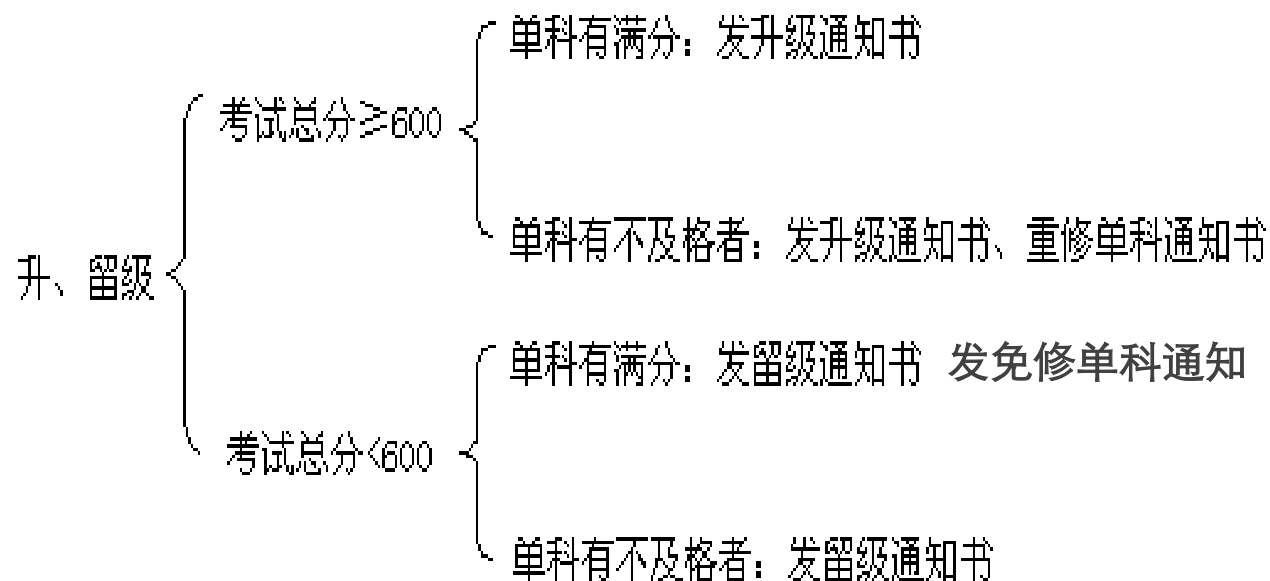
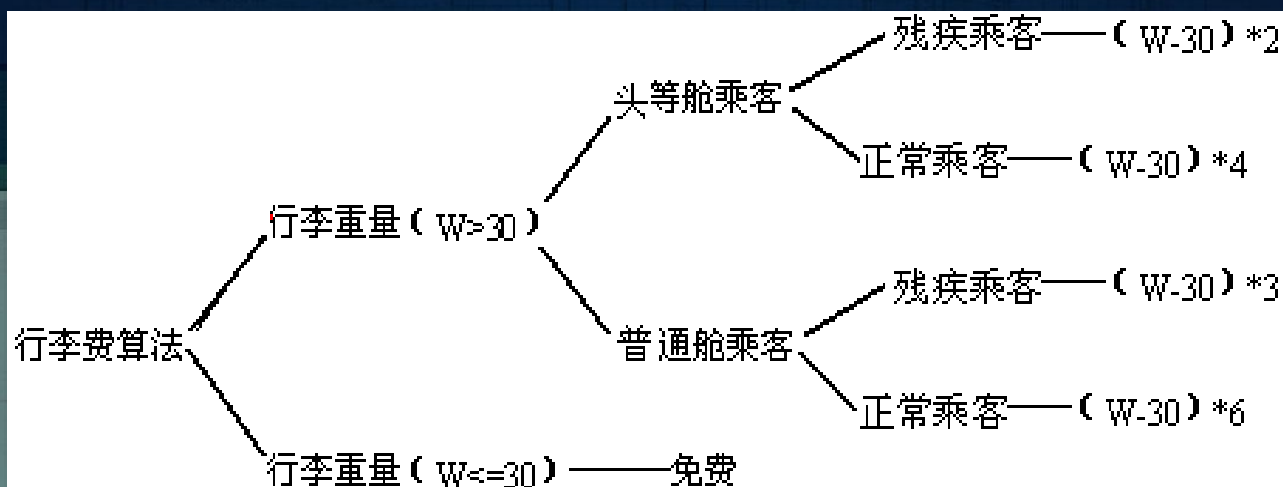
条件组合		1	2	3	4	5	6	7	8
条件	W>30公斤	√	√	√	√				
	头等舱乘客	√	√			√	√		
	残疾乘客	√		√		√		√	
行动	(W-30) *2	√							
	(W-30) *4		√						
	(W-30) *3			√					
	(W-30) *6				√				
	免费					√	√	√	√



例如，某校学籍管理升留级处理的判定表：

考试总分	≥ 600	≥ 600	< 600	< 600
单科	有满分	有不及格	有满分	有不及格
发升级通知书	Y	Y	N	N
发免修单科通知	N	N	Y	N
发留级通知书	N	N	Y	Y
发重修单科通知	N	Y	N	N





过程设计语言

- PDL（Process Design Language，简称PDL）是一种伪码设计语言。
- 伪码的语法规则分为“外语法”和“内语法”。
- PDL具有严格的**关键字外语法**（某种结构化的程序设计语言），用于定义控制结构和数据结构；同时它表示实际操作和条件的**内语法可使用自然语言的词汇**。



- PROCEDURE *spellcheck* IS
BEGIN

split document into single words

look up words in dictionary

display words which are not in dictionary

create a new dictionary

END *spellcheck*



小结

- 软件设计的目标是设计出所要开发的软件的模型，传统的软件工程方法学采用结构化设计技术完成软件设计
- 软件设计在软件工程的过程中处于技术核心地位，使软件开发过程中决定软件产品质量的关键阶段
- 软件设计必须依据对软件产品的需求来进行，因此结构化设计把结构化分析的结构作为基本输入信息。
- 为了获得高质量的软件设计结果，应该遵循模块化、抽象、逐步求精、信息隐藏和模块独立等基本设计原理



小结 (续)

- 通常可以使用层次图和结构图表示软件结构
- 面向数据流的设计方法是设计软件体系结构的一种系统化方法，它定义了一些映射规则，可以把数据流图变换成为软件的初步结构
- 人机界面设计是接口设计的一个组成部分
- 过程设计应该在数据设计、体系结构设计和接口设计完整之后进行，它是详细设计阶段的主要任务。



小结 (续)

- 数据设计：数据库逻辑模型、物理模型设计；
- 结构设计：层次图、HIPO图；
- 接口设计：系统间接口、系统内部模块间接口、人机交互接口；
- 过程设计：流程图、N-S图、PAD图、判定表、判定树、PDL。



The end of this chapter.



第4章 结构化设计

- 人机界面设计问题

- 系统响应时间
- 用户帮助设施
- 出错信息处理
- 命令交互



第4章 结构化设计

- 人机界面设计过程

原型+修改，迭代的过程



第4章 结构化设计

- 界面设计指南

- 一般交互

- 保持一致性

- 提供有意义的反馈

- 在执行有较大破坏性的动作之前要求用户确认

- 允许取消的操作

-



第4章 结构化设计

- 信息显示

只显示与当前工作内容有关的信息

不要用数据淹没用户

使用一致的标记、标准的缩写和可预知的颜色

允许用户保持可视化的语境

产生有意义的出错信息

使用窗口分割不同类型的信息

使用“模拟”显示方式表达信息

.....



第4章 结构化设计

- 数据输入

尽量减少用户的输入动作

保持信息显示和信息输入之间的一致性

允许用户自定义输入

交互应该灵活，可调整成为用户喜欢的输入方式

让用户控制交流

对所有输入动作都提供帮助

消除冗余的输入

